UNIVERSIDAD DON BOSCO	**UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN**	**CICLO 02-2026**
	MATERIA | **DESARROLLO DE SOFTWARE EMPRESARIAL**	**GUIA DE LABORATORIO #10**
	PRÁCTICA | **AUTENTICACION Y AUTORIZACION EN .NET**	

I. OBJETIVOS

- Que el estudiante instale y configure ASP.NET Identity en un proyecto ASP.NET Core.
- Que el estudiante implemente la autenticación y autorización utilizando Identity para proteger recursos en una aplicación web.
- Que el estudiante defina y gestione roles y usuarios, implementando políticas de autorización basadas en roles y claims.
- Que el estudiante configure autenticación externa con proveedores como Google o Facebook en una aplicación ASP.NET Core.
- Que el estudiante utilice el flujo de autenticación y autorización para implementar control de acceso en una API.

II. INTRODUCCIÓN TEORICA

¿Qué es ASP.NET Core Identity?

ASP.NET Core Identity es un sistema de autenticación y administración de usuarios que proporciona una forma fácil de agregar funcionalidad de inicio de sesión y control de acceso a las aplicaciones ASP.NET Core. Identity se integra de manera profunda con ASP.NET Core, permitiendo gestionar usuarios, roles y credenciales de una manera robusta y extensible.

Historia y Origen de ASP.NET Core Identity

ASP.NET Identity se introdujo inicialmente en 2013 como una evolución de los sistemas de autenticación anteriores de ASP.NET, como Membership y Simple Membership. A medida que las aplicaciones web modernas se volvieron más complejas, la necesidad de un sistema de autenticación más flexible y extensible creció. Así nació ASP.NET Identity, que fue rediseñado completamente para integrarse con ASP.NET Core, con un enfoque en la modularidad, seguridad y extensibilidad.

Arquitectura y Funcionamiento de ASP.NET Core Identity

ASP.NET Core Identity se basa en un conjunto de interfaces y clases que permiten la implementación de características clave como la autenticación, la autorización y la administración de usuarios. A través de estas clases e interfaces, puedes personalizar el comportamiento de Identity según las necesidades de tu aplicación.

- **Autenticación**: ASP.NET Core Identity gestiona el proceso de inicio de sesión de los usuarios mediante la creación y validación de cookies de autenticación. También permite la integración con proveedores externos como Google, Facebook, Twitter, etc.
- **Autorización**: después de autenticar al usuario, Identity permite definir políticas de acceso basadas en roles o claims. Las políticas de autorización permiten controlar qué usuarios pueden acceder a qué partes de la aplicación.
- **Gestión de Usuarios y Roles**: ASP.NET Core Identity incluye la funcionalidad para crear, editar, y eliminar usuarios y roles. Los roles se pueden asignar a los usuarios para controlar el acceso a diferentes recursos de la aplicación.
- **Almacenamiento de Datos**: de manera predeterminada, ASP.NET Core Identity almacena los datos de usuarios y roles en una base de datos SQL Server utilizando Entity Framework Core. Sin embargo,

también es posible personalizar el almacenamiento para utilizar otros sistemas, como NoSQL o incluso servicios externos.

Autenticación y Autorización en ASP.NET Core

1. **Autenticación**: El proceso de autenticación consiste en verificar la identidad del usuario que intenta acceder a un recurso. En ASP.NET Core, esto se realiza generalmente a través de cookies o tokens. Una vez autenticado, se genera una cookie de sesión que se utiliza para identificar al usuario en solicitudes posteriores.

2. **Autorización**: Una vez autenticado, el sistema verifica si el usuario tiene los permisos necesarios para acceder al recurso solicitado. ASP.NET Core Identity utiliza roles y claims para definir políticas de autorización que permiten o restringen el acceso a controladores o acciones específicas en la aplicación.

Usos Comunes de ASP.NET Core Identity

- **Aplicaciones Web Empresariales**: para gestionar usuarios y permisos en aplicaciones de gran escala.
- **APIs Seguras**: proteger endpoints en APIs REST utilizando autenticación basada en tokens o cookies.
- **Portales y Sistemas Internos**: donde la administración de usuarios y roles es una necesidad crítica para asegurar diferentes niveles de acceso a información sensible.

III. DESARROLLO DE LA PRÁCTICA

Parte 1 – Proyecto Web API

1. Cree un nuevo proyecto de tipo Web API para la gestión de vehículos.

2. Agregar las clases **Marca**, **Modelo**, **Vehículo** de acuerdo con las siguientes especificaciones.

```
public class Marca
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;

    public ICollection<Modelo> Modelos { get; set; } = new List<Modelo>();
}

public class Modelo
{
    public int Id { get; set; }
    public int MarcaId { get; set; }
    public string Nombre { get; set; } = string.Empty;

    public Marca Marca { get; set; } = null!;
    public ICollection<Vehiculo> Vehiculos { get; set; } = new List<Vehiculo>();
}

public class Vehiculo
{
    public int Id { get; set; }
    public int ModeloId { get; set; }
    public int Anio { get; set; }

    public Modelo Modelo { get; set; } = null!;
}
```

3. Configure el proyecto para incluir los paquetes necesarios de Entity Framework para el funcionamiento con SQL Server.

- **Microsoft.EntityFrameworkCore**
- **Microsoft.EntityFrameworkCore.Tools**
- **Microsoft.EntityFrameworkCore.SqlServer**

4. Agregue la clase **ApplicationDbContext** y modifíquela como se muestra a continuación.

```
public class ApplicationDbContext(DbContextOptions options) : DbContext(options)
{
    public DbSet<Marca> Marcas { get; set; }
    public DbSet<Modelo> Modelos { get; set; }
    public DbSet<Vehiculo> Vehiculos { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ==========================
        // MARCAS
        // ==========================
        modelBuilder.Entity<Marca>().HasData(
            new Marca
            {
                Id = 1,
                Nombre = "Toyota"
            },
            new Marca
            {
                Id = 2,
                Nombre = "Honda"
            },
            new Marca
            {
                Id = 3,
                Nombre = "Ford"
            },
            new Marca
            {
                Id = 4,
                Nombre = "Chevrolet"
            }
        );

        // ==========================
        // MODELOS
        // ==========================
        modelBuilder.Entity<Modelo>().HasData(
            new Modelo
            {
                Id = 1,
                MarcaId = 1,
                Nombre = "Corolla"
            },
            new Modelo
            {
                Id = 2,
```

```
            MarcaId = 1,
            Nombre = "Hilux"
        },
        new Modelo
        {
            Id = 3,
            MarcaId = 2,
            Nombre = "Civic"
        },
        new Modelo
        {
            Id = 4,
            MarcaId = 2,
            Nombre = "CR-V"
        },
        new Modelo
        {
            Id = 5,
            MarcaId = 3,
            Nombre = "Mustang"
        },
        new Modelo
        {
            Id = 6,
            MarcaId = 3,
            Nombre = "Ranger"
        },
        new Modelo
        {
            Id = 7,
            MarcaId = 4,
            Nombre = "Camaro"
        },
        new Modelo
        {
            Id = 8,
            MarcaId = 4,
            Nombre = "Silverado"
        }
    );

    // ==========================
    // VEHÍCULOS
    // ==========================
    modelBuilder.Entity<Vehiculo>().HasData(
        new Vehiculo
        {
            Id = 1,
            ModeloId = 1,
            Anio = 2020
        },
        new Vehiculo
        {
            Id = 2,
            ModeloId = 1,
            Anio = 2022
        },
        new Vehiculo
        {
            Id = 3,
            ModeloId = 2,
```

```
                Anio = 2021
            },
            new Vehiculo
            {
                Id = 4,
                ModeloId = 3,
                Anio = 2019
            },
            new Vehiculo
            {
                Id = 5,
                ModeloId = 3,
                Anio = 2023
            },
            new Vehiculo
            {
                Id = 6,
                ModeloId = 4,
                Anio = 2022
            },
            new Vehiculo
            {
                Id = 7,
                ModeloId = 5,
                Anio = 2021
            },
            new Vehiculo
            {
                Id = 8,
                ModeloId = 6,
                Anio = 2020
            },
            new Vehiculo
            {
                Id = 9,
                ModeloId = 7,
                Anio = 2018
            },
            new Vehiculo
            {
                Id = 10,
                ModeloId = 8,
                Anio = 2023
            }
        );
    }
}
```

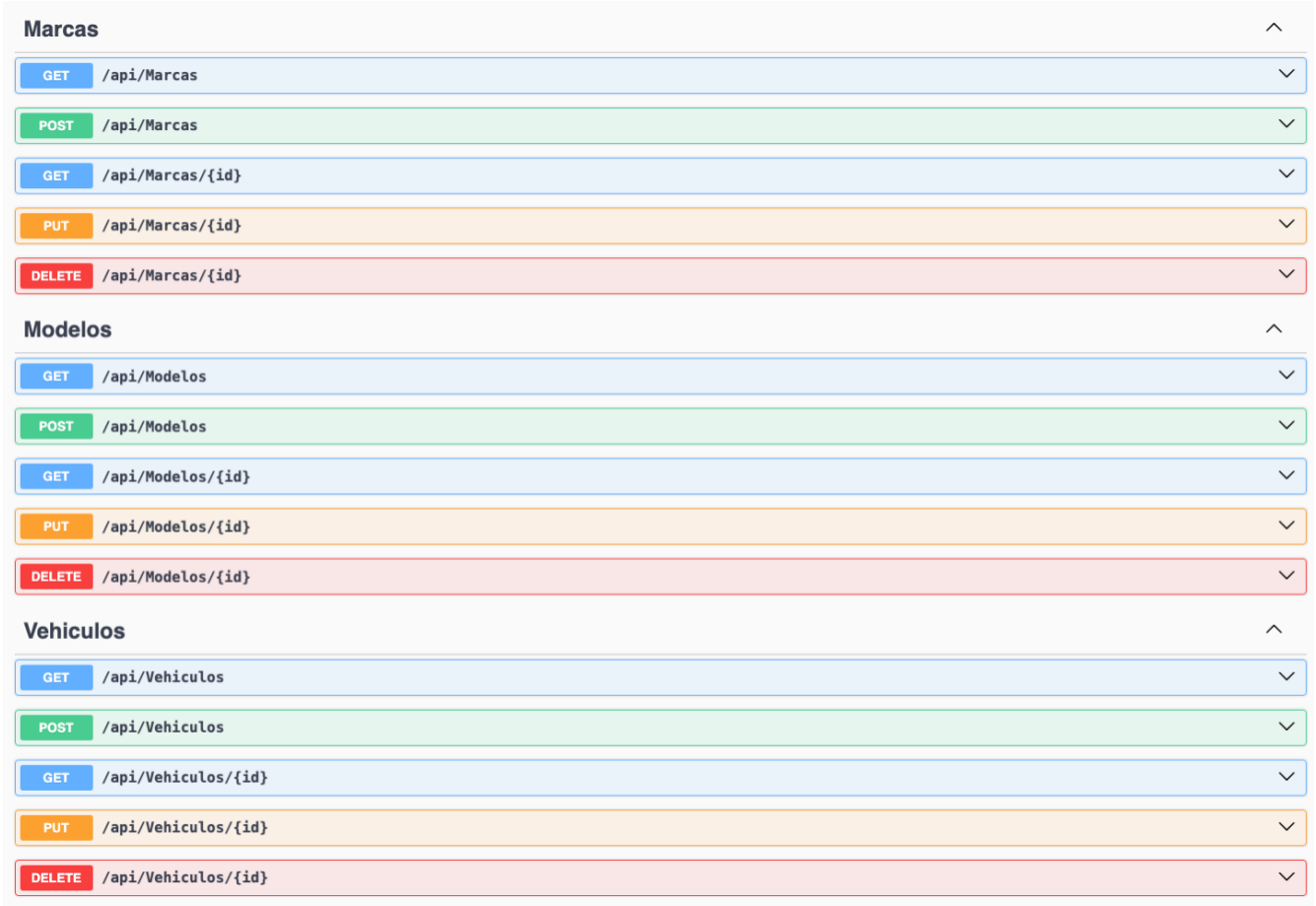
6. Configure en la clase **Program** en contexto de la base de datos.

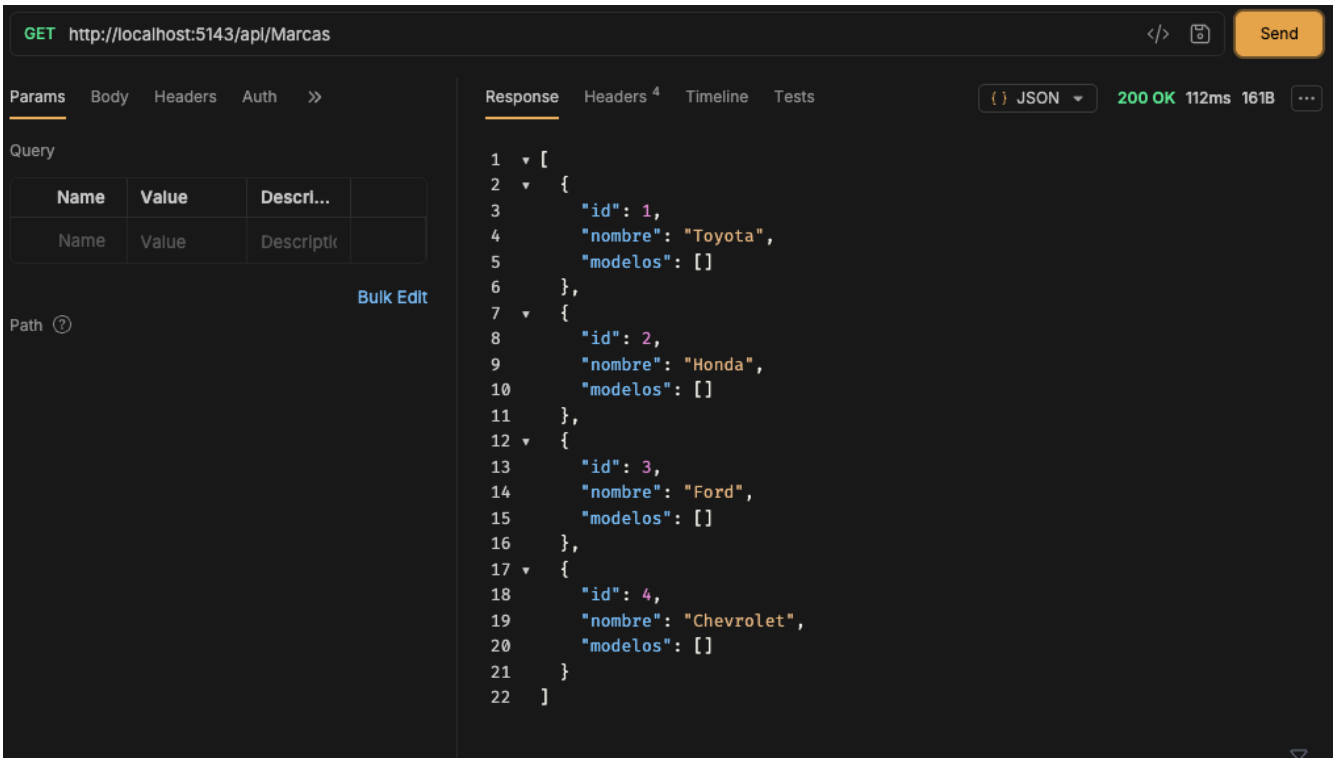
```
builder.Services.AddDbContext<ApplicationDbContext>(options =>

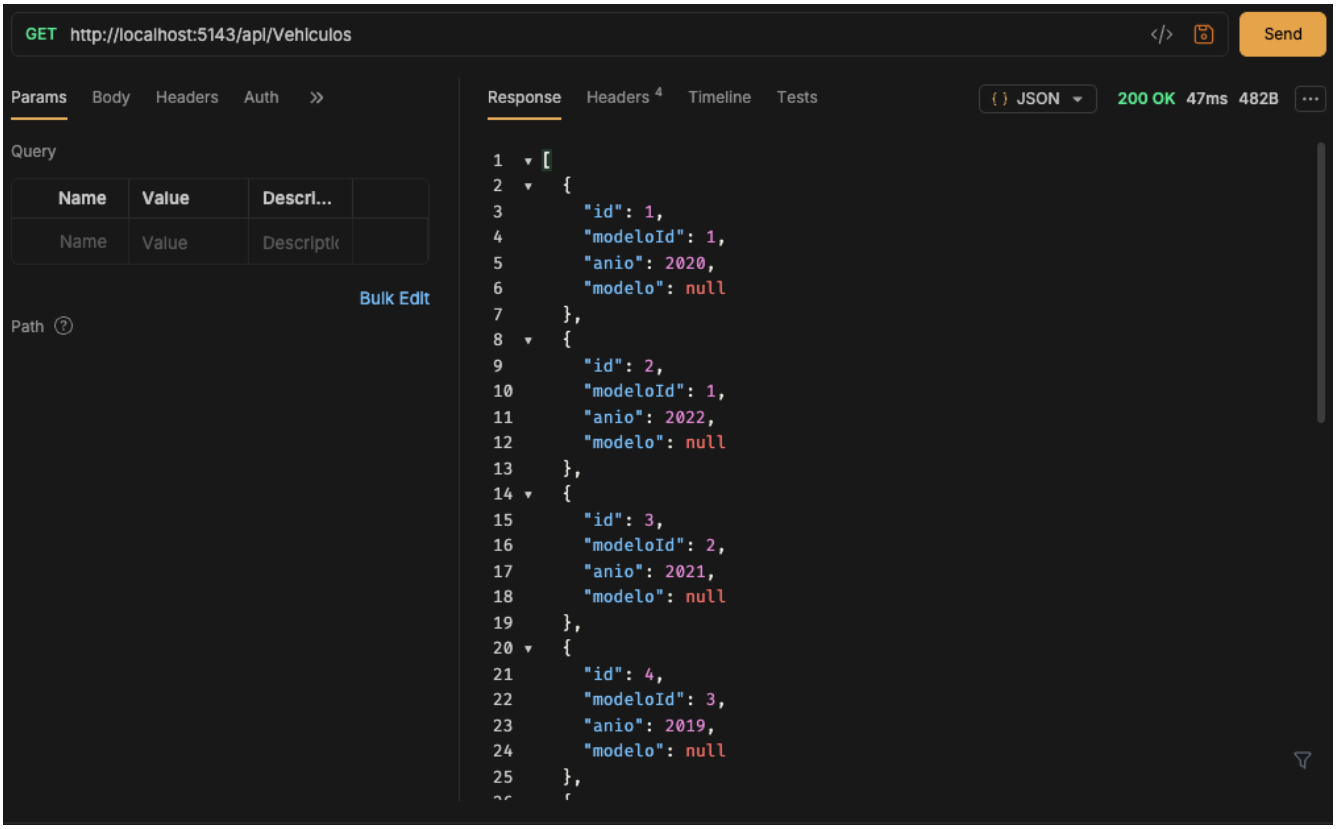
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")
));
```

7. Agregue una nueva migración y actualice la base de datos.

8. Agregue los controladores haciendo uso de Scaffolding para cada modelo y compruebe el funcionamiento completo de la API.







Parte 2 – Configuración ASP.NET Core Identity

1. Instale el paquete **Microsoft.AspNetCore.Identity.EntityFrameworkCore**, asegúrese de utilizar una versión compatible con la versión de .NET de su proyecto**.**

2. En la carpeta **Models**, agregue una nueva clase con el nombre **Usuario** y modifique el código como se muestra a continuación.

```
public class Usuario : IdentityUser
{

}
```

- En .NET Core, la clase **IdentityUser** es parte del sistema de ASP.NET Core Identity, que proporciona una implementación completa para la gestión de usuarios, autenticación y autorización en aplicaciones web.
- **IdentityUser** representa a un usuario en el sistema de identidad y contiene la información básica necesaria para manejar la autenticación y la gestión de cuentas de usuario.

3. Modifique la clase **Program** para incluir los servicios de autenticación y autorización.

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")
));

builder.Services.AddControllers();
```

```
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// INCIO -> AUTENTICACION Y AUTORIZACION
builder.Services.AddAuthorization();
builder.Services.AddAuthentication()
    .AddCookie(IdentityConstants.ApplicationScheme);
builder.Services.AddIdentityCore<Usuario>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddApiEndpoints();

// FIN -> AUTENTICACION Y AUTORIZACION

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthentication(); // NUEVO

app.UseAuthorization(); // NUEVO

app.MapControllers();

app.MapIdentityApi<Usuario>(); // NUEVO

app.Run();
```

4. Modifique el **ApplicationDbContext** como se muestra a continuacion.

```
public class ApplicationDbContext(DbContextOptions options) :
IdentityDbContext<Usuario>(options)
```

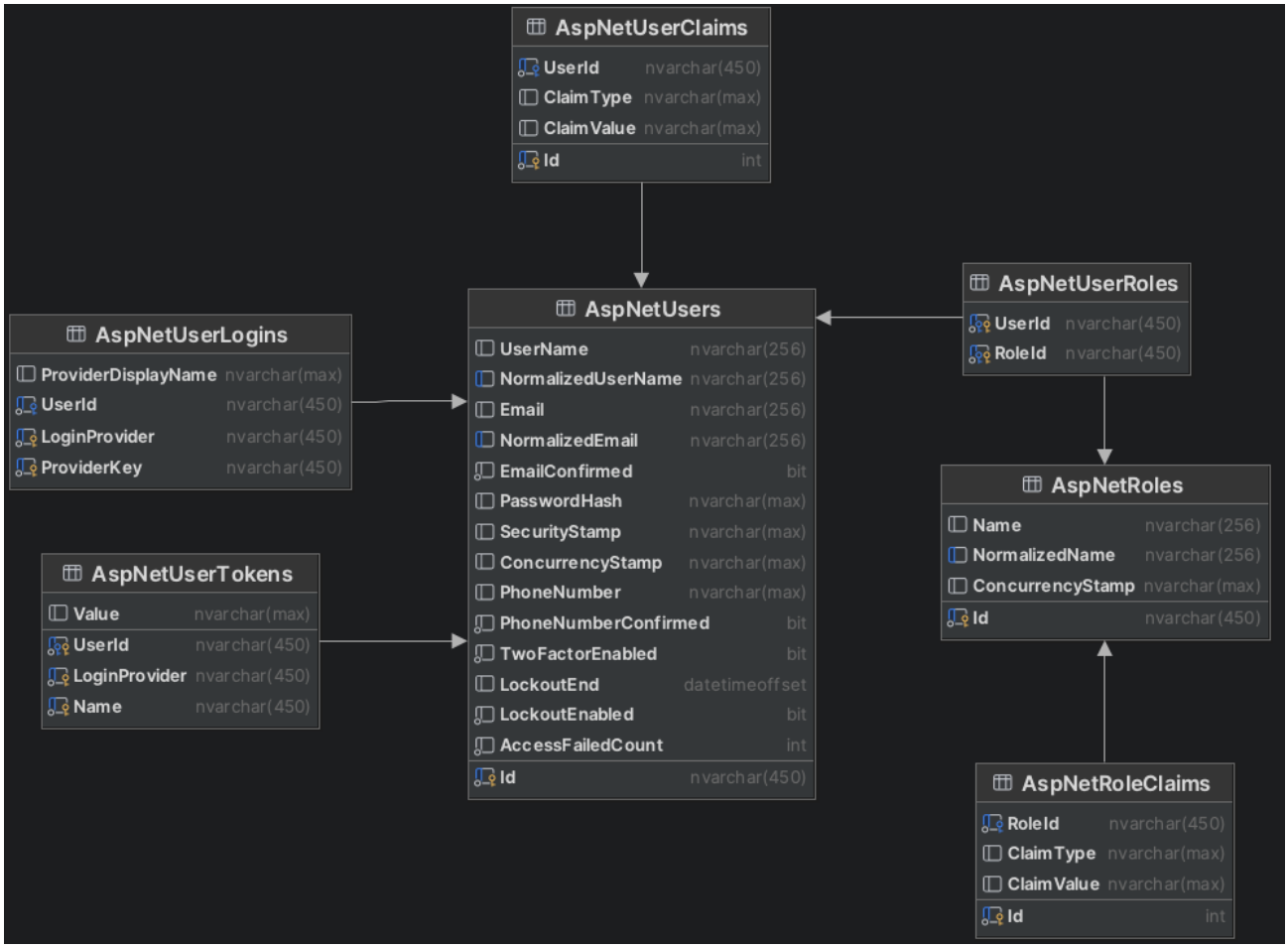
- **DbContext** es la clase base de Entity Framework Core que permite trabajar con la base de datos.
- **IdentityDbContext<Usuario>** también es un **DbContext**, pero está especializado para ASP.NET Core Identity.
- Al utilizar IdentityDbContext<Usuario>, además de poder manejar las entidades propias de la aplicación, se obtiene toda la estructura necesaria para administrar usuarios, roles, contraseñas, claims y otros elementos relacionados con la autenticación y autorización.

5. Agregue una nueva migración y observe el archivo generado. La migración debe reflejar las siguientes tablas:

- **AspNetRoles**: almacena los roles (perfiles o grupos) que se pueden asignar a los usuarios, como **Admin** o **User**.
- **AspNetUsers**: contiene la información de los usuarios, como el nombre de usuario, el correo electrónico, la contraseña encriptada, y otros datos del perfil.

- **AspNetRoleClaims**: asocia **claims** (afirmaciones o atributos) a los roles, lo que permite asignar permisos o información adicionales a un rol específico.
- **AspNetUserClaims**: almacena las **claims** relacionadas con los usuarios, lo que permite agregar información adicional a un usuario, como permisos o atributos personalizados.
- **AspNetUserLogins**: almacena los inicios de sesión externos asociados a un usuario (por ejemplo, si inician sesión con Google, Facebook, etc.).
- **AspNetUserRoles**: tabla de relación que une a los usuarios con los roles. Indica qué rol tiene asignado cada usuario.
- **AspNetUserTokens**: almacena tokens de autenticación, como los usados en la autenticación por dos factores (2FA) o para mantener sesiones abiertas en un dispositivo.

6. Aplique la migración y confirme mediante el explorador de la base de datos la creación de las tablas.



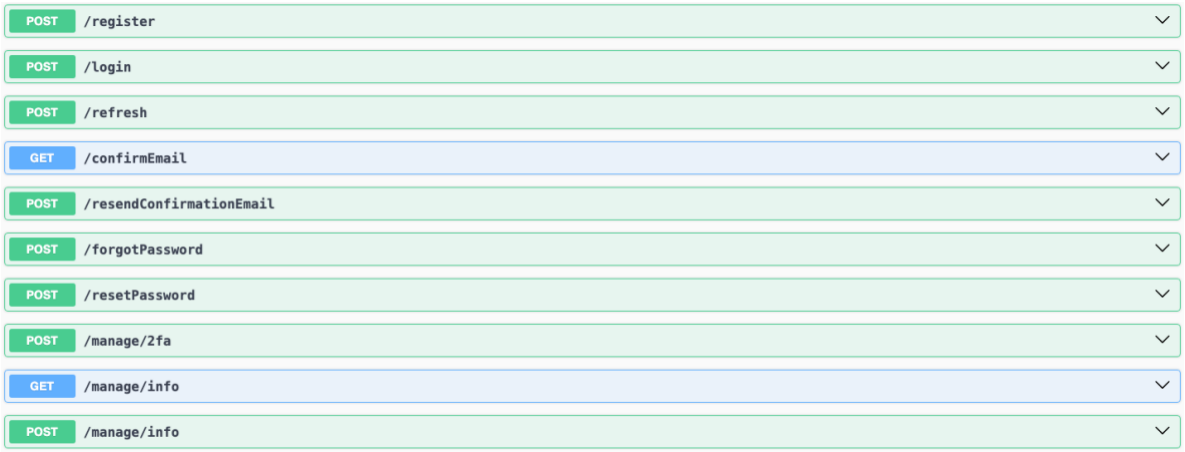
7. Modifique los controladores **MarcasController**, **ModelosController** y **VehiculosController**, agregando el atributo **[Authorize]** como se muestra a continuación.

```
[Route("api/[controller]")]
[ApiController]
[Authorize] // NUEVO ATRIBUTO
public class MarcasController : ControllerBase
```

```
[Route("api/[controller]")]
[ApiController]
[Authorize] // NUEVO ATRIBUTO
public class ModelosController : ControllerBase
```

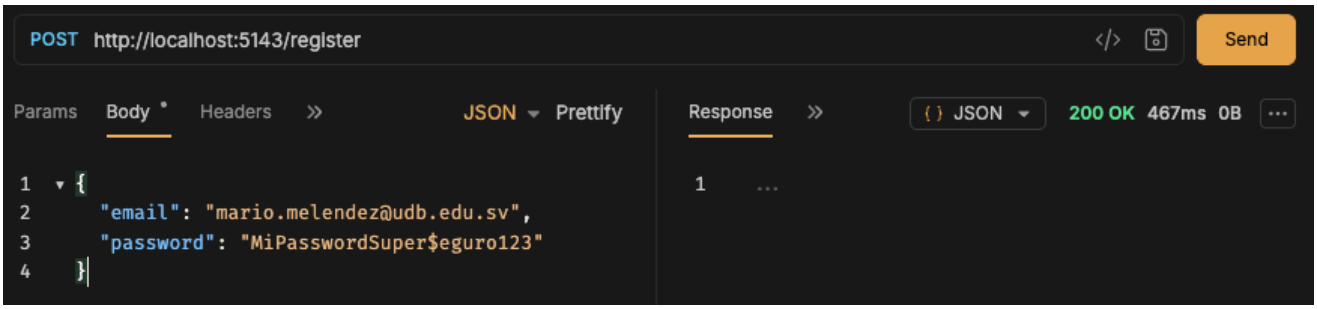
```
[Route("api/[controller]")]
[ApiController]
[Authorize] // NUEVO ATRIBUTO
public class VehiculosController : ControllerBase
```

8. Ejecute el proyecto y observe que aparecen nuevos endpoints correspondientes al modelo **Usuario**.



9. Intente ejecutar los endpoints de los modelos **Marcas**, **Modelos** y **Vehículos**. Observe el mensaje de respuesta es un error, debido a que no se encuentra autenticados en la API.

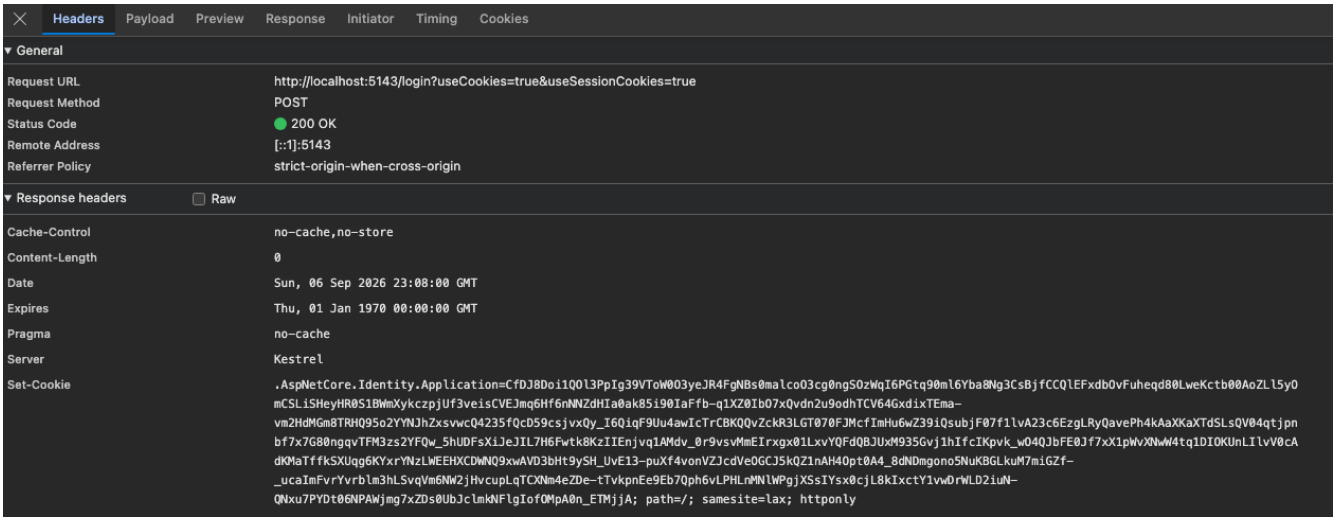
10. Haga uso del endpoint **POST: /register** para agregar un nuevo usuario.



11. Haciendo uso del endpoint **POST: /login?useCookies=true&useSessionCookies=true** inicie sesión con el usuario recién creado.



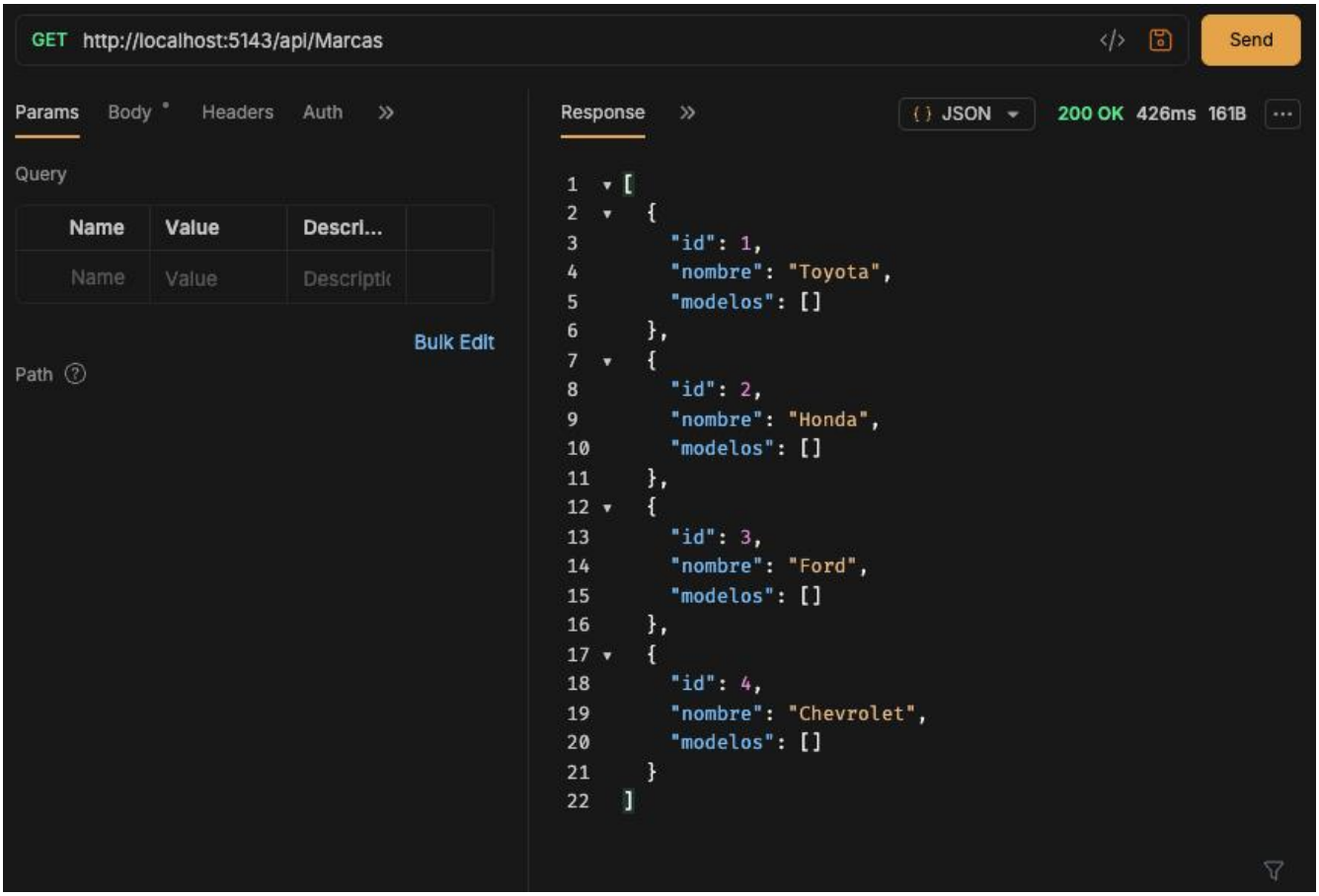
- El endpoint /login no retorna nada, pero se encarga de almacenar una cookie válida para poder consultar los demás métodos.
- Si está haciendo uso de Swagger la cookie se puede ver en las herramientas de desarrollador del navegador.



- Si utiliza otro cliente Http como bruno, explore la sección de cookies para ver el valor almacenado.



12. Compruebe el funcionamiento de los endpoints ahora que ya se encuentra autenticado.



IV. DESARROLLO DE HABILIDADES

Realice una investigación y la implementación en el proyecto de esta guía sobre cómo utilizar roles y segmentar el acceso a los controladores usando el paquete **Microsoft.AspNetCore.Identity.EntityFrameworkCore**